

Лабораторная работа №2.

Первоначальная настройка git

Дисциплина: Архитектура компьютеров и операционные системы

Игнатенко Денис Беньяминович

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	9
4.1	Установка программного обеспечения	9
4.2	Базовая настройка git	10
4.3	Создание ключей SSH	11
4.4	Добавление SSH ключа на GitHub	13
4.5	Создание ключа PGP	13
4.6	Добавление PGP ключа на GitHub	15
4.7	Настройка автоматических подписей коммитов	16
4.8	Настройка gh (GitHub CLI)	16
4.9	Создание репозитория курса	17
5	Ответы на контрольные вопросы	19
6	Выводы	22
	Список литературы	23

Список иллюстраций

4.1	Установка git и gh	9
4.2	Проверка версий git и gh	10
4.3	Базовая настройка git	11
4.4	Генерация RSA ключа	11
4.5	Генерация Ed25519 ключа	12
4.6	Проверка созданных SSH ключей	12
4.7	Вывод публичного ключа ed25519	12
4.8	Добавление SSH ключа на GitHub	13
4.9	Генерация PGP ключа	14
4.10	Просмотр списка PGP ключей	15
4.11	Экспорт PGP ключа	15
4.12	Добавление GPG ключа на GitHub	15
4.13	Настройка автоподписи коммитов	16
4.14	Авторизация gh и проверка статуса	16
4.15	Создание каталога для курса	17
4.16	Создание репозитория на основе шаблона	17
4.17	Клонирование репозитория	17
4.18	Настройка каталога курса	18
4.19	Репозиторий на GitHub	18

Список таблиц

3.1	Основные команды git {#tbl:git-commands}	7
4.1	Команды настройки git config {#tbl:git-config}	10
4.2	Команды SSH и GPG {#tbl:ssh-gpg}	13

1 Цель работы

Изучить идеологию и применение средств контроля версий. Освоить умения по работе с git.

2 Задание

- Создать базовую конфигурацию для работы с git
- Создать ключ SSH
- Создать ключ PGP
- Настроить подписи git
- Зарегистрироваться на Github
- Создать локальный каталог для выполнения заданий по предмету

3 Теоретическое введение

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется.

Git — распределённая система контроля версий. В отличие от классических систем, в распределённых VCS центральный репозиторий не является обязательным — каждый разработчик имеет полную копию репозитория локально.

Основные команды git представлены в таблице ??.

Таблица 3.1: Основные команды git {#tbl:git-commands}

Команда	Описание
git init	Инициализация репозитория
git clone <url>	Клонирование репозитория
git add .	Добавить все изменения в индекс
git commit -m "msg"	Зафиксировать изменения
git push	Отправить на сервер
git pull	Получить с сервера
git status	Статус рабочего каталога
git log	История коммитов

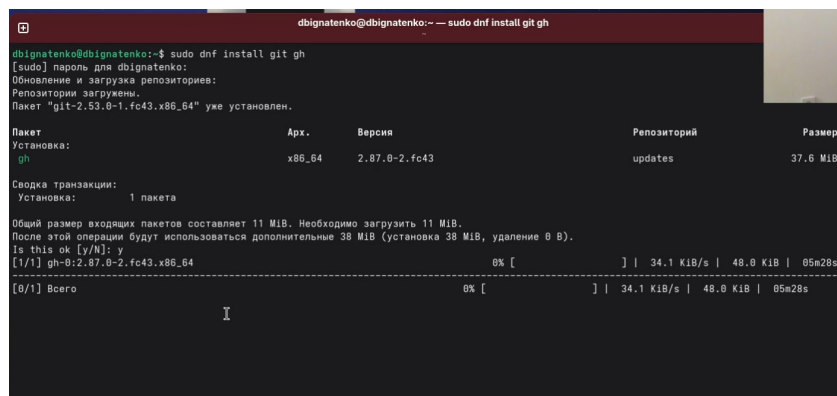
Команда	Описание
<code>git branch</code>	Список веток
<code>git checkout -b branch</code>	Создать и переключиться на ветку
<code>git merge branch</code>	Слить ветку

4 Выполнение лабораторной работы

4.1 Установка программного обеспечения

Устанавливаю git и gh (GitHub CLI) с помощью пакетного менеджера dnf (рис. 4.1).

```
sudo dnf install git gh
```



```
dbignatenko@dbignatenko:~$ sudo dnf install git gh
[sudo] пароль для dbignatenko:
Обновление и загрузка репозитория:
Репозитории загружены.
Пакет "git-2.53.0-1.fc43.x86_64" уже установлен.

Пакет
Установка:
  Арх.      Версия      Репозиторий      Размер
  gh        x86_64      2.87.0-2.fc43    updates          37.6 MiB

Сводка транзакции:
  Установка:      1 пакета

Общий размер входящих пакетов составляет 11 MiB. Необходимо загрузить 11 MiB.
После этой операции будут использоваться дополнительные 38 MiB (установка 38 MiB, удаление 0 B).
Is this ok [y/N]: y
[1/1] gh-0:2.87.0-2.fc43.x86_64                                0% [          ] | 34.1 KiB/s | 48.0 KiB | 05m28s
[0/1] Всего                                                    0% [          ] | 34.1 KiB/s | 48.0 KiB | 05m28s
```

Рис. 4.1: Установка git и gh

Проверяю корректность установки, выводя версии установленных программ (рис. 4.2).

```
git --version
```

```
gh --version
```

```
dbignatenko@dbignatenko:~$ git --version
git version 2.53.0
dbignatenko@dbignatenko:~$ gh --version
gh version 2.87.0 (2026-02-18)
https://github.com/cli/cli/releases/tag/v2.87.0
dbignatenko@dbignatenko:~$
```

Рис. 4.2: Проверка версий git и gh

4.2 Базовая настройка git

Выполняю базовую настройку git: задаю имя и email владельца репозитория, настраиваю utf-8 в выводе сообщений, задаю имя начальной ветки master, настраиваю параметры autocrlf и safecrlf (рис. 4.3).

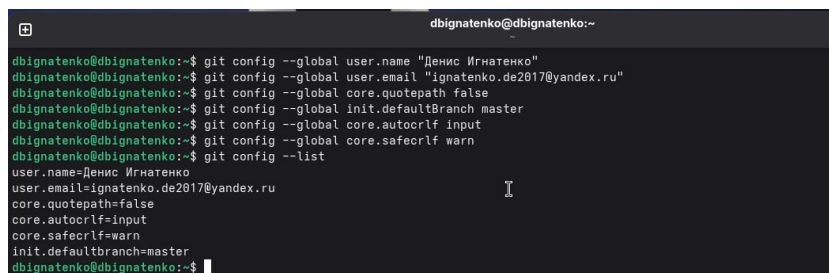
Команды настройки git config представлены в таблице ??.

Таблица 4.1: Команды настройки git config {#tbl:git-config}

Команда	Описание
git config --global user.name	Имя пользователя
git config --global user.email	Email пользователя
git config --global core.quotepath false	UTF-8 в выводе
git config --global init.defaultBranch master	Имя ветки по умолчанию
git config --global core.autocrlf input	Конвертация CRLF→LF
git config --global core.safecrlf warn	Предупреждение о конвертации
git config --global user.signingkey	Ключ для подписи
git config --global commit.gpgsign true	Автоподпись коммитов

```
git config --global user.name "Денис Игнатенко"
```

```
git config --global user.email "ignatenko.de2017@yandex.ru"
git config --global core.quotepath false
git config --global init.defaultBranch master
git config --global core.autocrlf input
git config --global core.safecrlf warn
git config --list
```



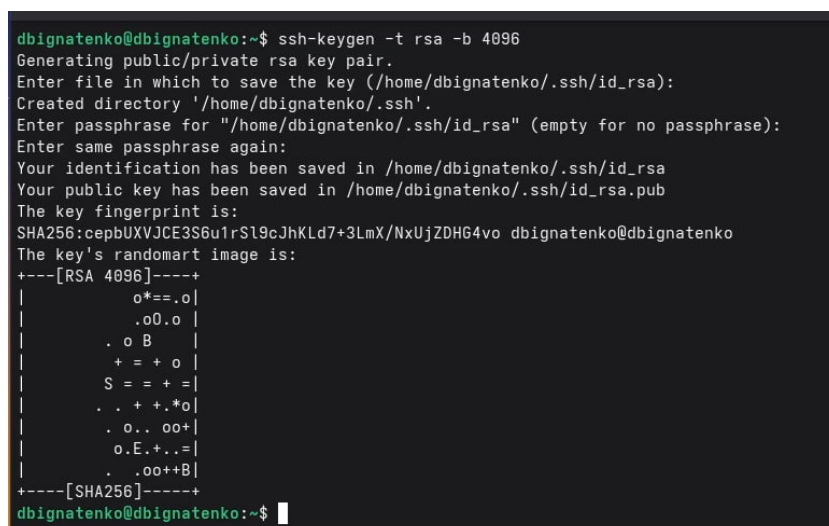
```
dbignatenko@dbignatenko:~$ git config --global user.name "Денис Игнатенко"
dbignatenko@dbignatenko:~$ git config --global user.email "ignatenko.de2017@yandex.ru"
dbignatenko@dbignatenko:~$ git config --global core.quotepath false
dbignatenko@dbignatenko:~$ git config --global init.defaultBranch master
dbignatenko@dbignatenko:~$ git config --global core.autocrlf input
dbignatenko@dbignatenko:~$ git config --global core.safecrlf warn
dbignatenko@dbignatenko:~$ git config --list
user.name=Денис Игнатенко
user.email=ignatenko.de2017@yandex.ru
core.quotepath=false
core.autocrlf=input
core.safecrlf=warn
init.defaultbranch=master
dbignatenko@dbignatenko:~$
```

Рис. 4.3: Базовая настройка git

4.3 Создание ключей SSH

Создаю SSH ключ по алгоритму RSA с размером 4096 бит (рис. 4.4).

```
ssh-keygen -t rsa -b 4096
```



```
dbignatenko@dbignatenko:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/dbignatenko/.ssh/id_rsa):
Created directory '/home/dbignatenko/.ssh'.
Enter passphrase for "/home/dbignatenko/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/dbignatenko/.ssh/id_rsa
Your public key has been saved in /home/dbignatenko/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:cepBUXVJCE3S6u1rS19cJhKld7+3LmX/NxUjZDH64vo dbignatenko@dbignatenko
The key's randomart image is:
+---[RSA 4096]-----+
|      o*==.o|
|      .oO.o |
|      . o B  |
|      + = + o |
|      S = + = |
|      . . + . * o |
|      . o.. oo+ |
|      o.E.+..= |
|      . .oo++B|
+---[SHA256]-----+
dbignatenko@dbignatenko:~$
```

Рис. 4.4: Генерация RSA ключа

Создаю SSH ключ по алгоритму Ed25519 (рис. 4.5).

```
ssh-keygen -t ed25519
```

```
dbignatenko@dbignatenko:~$ ssh-keygen -t ed25519
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/dbignatenko/.ssh/id_ed25519):
Enter passphrase for "/home/dbignatenko/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/dbignatenko/.ssh/id_ed25519
Your public key has been saved in /home/dbignatenko/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:YwBkbDnWf+TGV+MIivY1NiM6/maUDQT6N3C594Em/bQ dbignatenko@dbignatenko
The key's randomart image is:
+--[ED25519 256]--+
|
|o+o..|
|..*o+ +.0 o|
|o..oo*o0 = + .|
|..o++o=.o .|
|...S*+.o|
|..oo*+. o|
|..o E|
|..+|
|..o|
+----[SHA256]-----+
dbignatenko@dbignatenko:~$
```

Рис. 4.5: Генерация Ed25519 ключа

Проверяю, что ключи созданы (рис. 4.6).

```
ls ~/.ssh/
```

```
dbignatenko@dbignatenko:~$ ls ~/.ssh/
id_ed25519 id_ed25519.pub id_rsa id_rsa.pub
dbignatenko@dbignatenko:~$
```

Рис. 4.6: Проверка созданных SSH ключей

Вывожу публичный ключ ed25519 для добавления на GitHub (рис. 4.7).

```
cat ~/.ssh/id_ed25519.pub
```

```
dbignatenko@dbignatenko:~$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZD11NTE5AAAAIN4w/sbqgSUPDqbFbnIkk+UkZ7ZuW61Mt1Nh2z+TsSdf dbignatenko@dbignatenko
dbignatenko@dbignatenko:~$
```

Рис. 4.7: Вывод публичного ключа ed25519

4.4 Добавление SSH ключа на GitHub

Перехожу в настройки GitHub (Settings → SSH and GPG keys), нажимаю «New SSH key», добавляю ключ с названием «LabKey» (рис. 4.8).

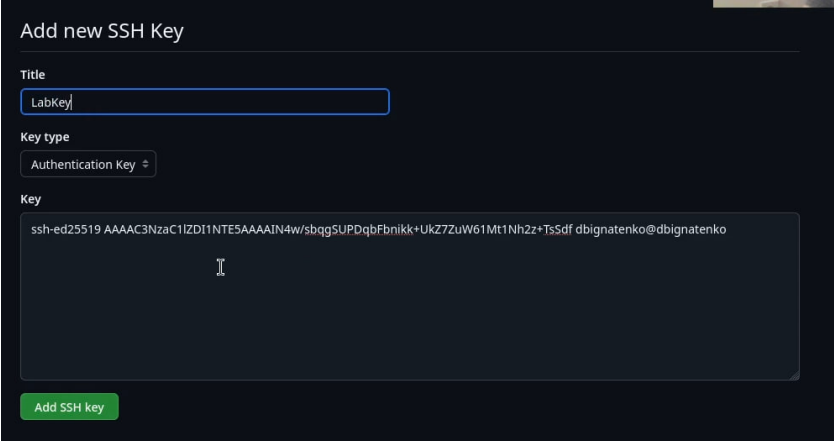


Рис. 4.8: Добавление SSH ключа на GitHub

4.5 Создание ключа PGP

Генерирую PGP ключ командой `gpg --full-generate-key`. Выбираю тип RSA and RSA, размер 4096 бит, срок действия 0 (не истекает). Ввожу имя и email (рис. 4.9).

Команды SSH и GPG представлены в таблице ??.

Таблица 4.2: Команды SSH и GPG {#tbl:ssh-gpg}

Команда	Описание
<code>ssh-keygen -t rsa -b 4096</code>	Генерация RSA ключа 4096 бит
<code>ssh-keygen -t ed25519</code>	Генерация Ed25519 ключа
<code>cat ~/.ssh/id_ed25519.pub</code>	Вывод публичного ключа
<code>gpg --full-generate-key</code>	Генерация PGP ключа
<code>gpg --list-secret-keys</code>	Список секретных ключей
<code>--keyid-format LONG</code>	

Команда	Описание
<code>gpg --armor --export</code> <code><fingerprint></code>	Экспорт PGP ключа
<code>gh auth login</code>	Авторизация GitHub CLI
<code>gh auth status</code>	Проверка статуса авторизации
<code>gh repo create</code>	Создание репозитория

`gpg --full-generate-key`

```
dbignatenko@dbignatenko:~$ gpg --full-generate-key
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: создан каталог '/home/dbignatenko/.gnupg'
Выберите тип ключа:
  (1) RSA and RSA
  (2) DSA and Elgamal
  (3) DSA (sign only)
  (4) RSA (sign only)
  (9) ECC (sign and encrypt) *default*
 (10) ECC (только для подписи)
 (14) Existing key from card
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
Запрошенный размер ключа - 4096 бит
Выберите срок действия ключа.
  0 = не ограничен
  <n> = срок действия ключа - n дней
  <n>w = срок действия ключа - n недель
  <n>m = срок действия ключа - n месяцев
  <n>y = срок действия ключа - n лет
Срок действия ключа? (0) 0
Срок действия ключа не ограничен
Все верно? (y/N) y

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: Денис
Адрес электронной почты: ignatenko.de2017@yandex.ru
```

Рис. 4.9: Генерация PGP ключа

Просматриваю список секретных ключей и копирую fingerprint (рис. 4.10).

`gpg --list-secret-keys --keyid-format LONG`

```
dbignatenko@dbignatenko:~$ gpg --list-secret-keys --keyid-format LONG
gpg: проверка таблицы доверия
gpg: marginals needed: 3 completes needed: 1 trust model: pgp
gpg: глубина: 0 достоверных: 1 подписанных: 0 доверие: 0-, 0q, 0n, 0m, 0f, 1u
[keyboard]
-----
sec   rsa4096/0A0A0C3B1EB935EC 2026-03-01 [SC]
      9C010FF2F05CF12A8DC7DD170A0A0C3B1EB935EC
uid   [ абсолютно ] Денис <ignatenko.de2017@yandex.ru>
ssb   rsa4096/8B5DFED849C476E3 2026-03-01 [E]
dbignatenko@dbignatenko:~$
```

Рис. 4.10: Просмотр списка PGP ключей

Экспортирую ключ в формате ASCII для добавления на GitHub (рис. 4.11).

```
gpg --armor --export 9C010FF2F05CF12A8DC7DD170A0A0C3B1EB935EC
```

```
dbignatenko@dbignatenko:~$ gpg --armor --export 9C010FF2F05CF12A8DC7DD170A0A0C3B1EB935EC
```

Рис. 4.11: Экспорт PGP ключа

4.6 Добавление PGP ключа на GitHub

Перехожу в настройки GitHub (Settings → SSH and GPG keys → New GPG key), вставляю экспортированный ключ (рис. 4.12).

Add new GPG key

Title

LabKey

Key

-----BEGIN PGP PUBLIC KEY BLOCK-----

mQINBGMkr6gBEADarzFNIJ3e4FR3btkhXjfsq0Ry8xvCm4JzPZLfNeUOXIH6UVMN
Im7I5yAhz9okhRB2sLFL+RmAjvXc34HvyHIMZJCS0teR1Y5uch19UqmKLA2FIKnWU
OGOb2dG6tsGpiAdnTXMQV9pLUuIoJdirfappH+WPrxTBw8Ach5fjq68fznA9Ri8
RtleO15ZB5xjhKos1yiUmHcFzLMMogigT4vOMIVEoLxYQ4pvl1x13NVufyUNWII6
rRqzN1nwqX+AaBUjcdSdAjoLAuyDcGk6sARWs+d1/oc8rjn51c2kovmCTQPlpPwx
riyxsXdBThHOe0FiuqdBWLKV5HBI3+o4Z53R0R2Hj2F/P3oEIRyCWM096RoilP9
wmHG2B6pn5Vg34PnAiY9yvhaZlptVLHv50tMaGHH2rmC8fnmjNmSJMDYL1J555

Add GPG key

Рис. 4.12: Добавление GPG ключа на GitHub

4.7 Настройка автоматических подписей коммитов

Настраиваю git для автоматической подписи коммитов с использованием созданного PGP ключа (рис. 4.13).

```
git config --global user.signingkey 9C010FF2F05CF12A8DC7DD170A0A0C3B1EB935EC
git config --global commit.gpgsign true
git config --global gpg.program $(which gpg2)
```



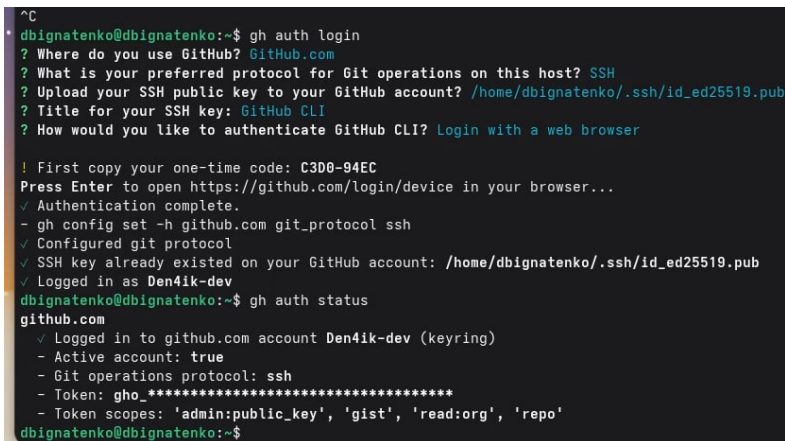
```
dbignatenko@dbignatenko:~$ git config --global user.signingkey 9C010FF2F05CF12A8DC7DD170A0A0C3B1EB935EC
dbignatenko@dbignatenko:~$ git config --global commit.gpgsign true
dbignatenko@dbignatenko:~$ git config --global gpg.program $(which gpg2)
```

Рис. 4.13: Настройка автоподписи коммитов

4.8 Настройка gh (GitHub CLI)

Авторизуюсь в GitHub CLI и проверяю статус авторизации (рис. 4.14).

```
gh auth login
gh auth status
```



```
dbignatenko@dbignatenko:~$ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? SSH
? Upload your SSH public key to your GitHub account? /home/dbignatenko/.ssh/id_ed25519.pub
? Title for your SSH key: GitHub CLI
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: C3D0-94EC
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol ssh
✓ Configured git protocol
✓ SSH key already existed on your GitHub account: /home/dbignatenko/.ssh/id_ed25519.pub
✓ Logged in as Den4ik-dev
dbignatenko@dbignatenko:~$ gh auth status
github.com
✓ Logged in to github.com account Den4ik-dev (keyring)
- Active account: true
- Git operations protocol: ssh
- Token: gho_*****
- Token scopes: 'admin:public_key', 'gist', 'read:org', 'repo'
dbignatenko@dbignatenko:~$
```

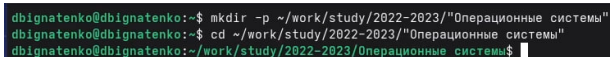
Рис. 4.14: Авторизация gh и проверка статуса

4.9 Создание репозитория курса

Создаю каталог для хранения репозитория курса (рис. 4.15).

```
mkdir -p ~/work/study/2022-2023/"Операционные системы"
```

```
cd ~/work/study/2022-2023/"Операционные системы"
```

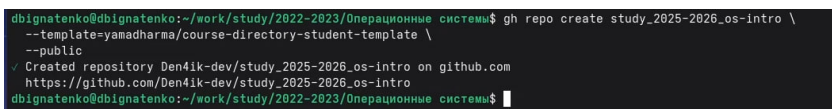


```
dbignatenko@dbignatenko:~$ mkdir -p ~/work/study/2022-2023/"Операционные системы"
dbignatenko@dbignatenko:~$ cd ~/work/study/2022-2023/"Операционные системы"
dbignatenko@dbignatenko:~/work/study/2022-2023/Операционные системы$
```

Рис. 4.15: Создание каталога для курса

Создаю репозиторий на основе шаблона yamadharm/course-directory-student-template (рис. 4.16).

```
gh repo create study_2025-2026_os-intro \
  --template=yamadharm/course-directory-student-template \
  --public
```

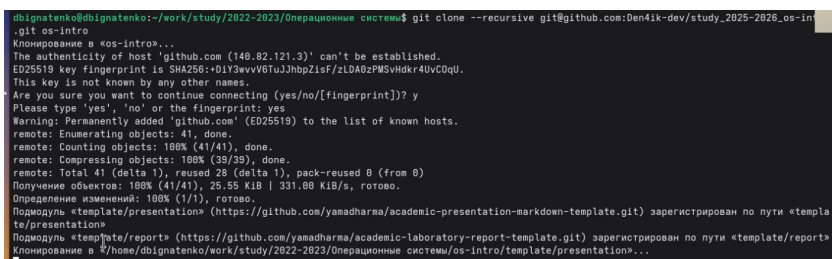


```
dbignatenko@dbignatenko:~/work/study/2022-2023/Операционные системы$ gh repo create study_2025-2026_os-intro \
  --template=yamadharm/course-directory-student-template \
  --public
✓ Created repository Den4ik-dev/study_2025-2026_os-intro on github.com
https://github.com/Den4ik-dev/study_2025-2026_os-intro
dbignatenko@dbignatenko:~/work/study/2022-2023/Операционные системы$
```

Рис. 4.16: Создание репозитория на основе шаблона

Клонирую созданный репозиторий (рис. 4.17).

```
git clone --recursive git@github.com:Den4ik-dev/study_2025-2026_os-intro.git os-intro
```



```
dbignatenko@dbignatenko:~/work/study/2022-2023/Операционные системы$ git clone --recursive git@github.com:Den4ik-dev/study_2025-2026_os-intro.git os-intro
Клонирование в «os-intro»...
The authenticity of host 'github.com (149.82.121.3)' can't be established.
ED25519 key fingerprint is SHA256:0dY3wvV6IuJhbpZisF/zL0A0zPMsvHdkr4UVC0Qu.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
remote: Enumerating objects: 41, done.
remote: Counting objects: 100% (41/41), done.
remote: Compressing objects: 100% (39/39), done.
remote: Total 41 (delta 1), reused 28 (delta 1), pack-reused 8 (from 0)
Получение объектов: 100% (41/41), 25.55 KiB | 331.00 KiB/s, готово.
Определение изменений: 100% (1/1), готово.
Подмодуль «template/presentation» (https://github.com/yamadharm/academic-presentation-markdown-template.git) зарегистрирован по пути «template/presentation»
Подмодуль «template/report» (https://github.com/yamadharm/academic-laboratory-report-template.git) зарегистрирован по пути «template/report»
Клонирование в «/home/dbignatenko/work/study/2022-2023/Операционные системы/os-intro/template/presentation»...
```

Рис. 4.17: Клонирование репозитория

Настраиваю каталог курса: удаляю лишние файлы, создаю файл COURSE и запускаю make (рис. 4.18).

```
cd os-intro
rm package.json
echo os-intro > COURSE
make
```

```
dbignatenko@dbignatenko:~/work/study/2025-2026/Операционные системы$ cd os-intro
dbignatenko@dbignatenko:~/work/study/2025-2026/Операционные системы/os-intro$ rm package.json
dbignatenko@dbignatenko:~/work/study/2025-2026/Операционные системы/os-intro$ echo os-intro > COURSE
dbignatenko@dbignatenko:~/work/study/2025-2026/Операционные системы/os-intro$ make
```

Рис. 4.18: Настройка каталога курса

Проверяю, что репозиторий успешно создан на GitHub (рис. 4.19).

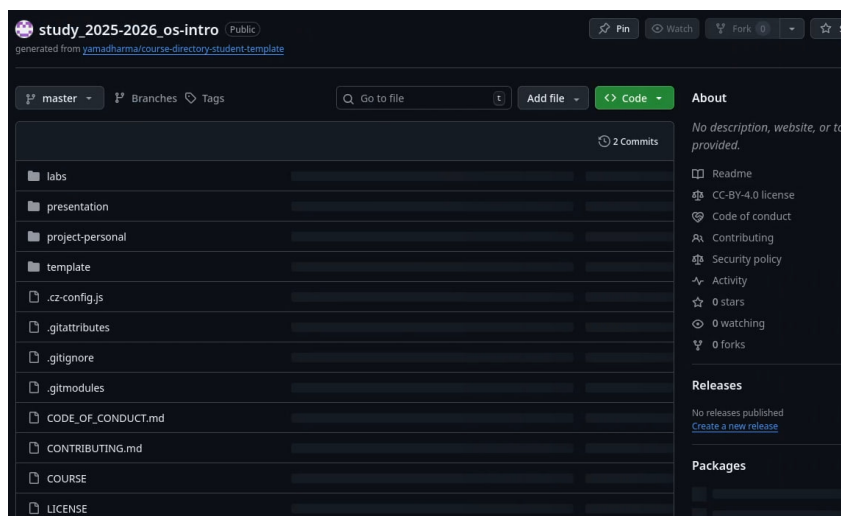


Рис. 4.19: Репозиторий на GitHub

5 Ответы на контрольные вопросы

1. Что такое системы контроля версий (VCS)?

VCS — программный инструмент для отслеживания изменений в файлах проекта. Позволяет фиксировать изменения, возвращаться к предыдущим версиям, работать нескольким разработчикам над одним проектом без конфликтов.

2. Хранилище, commit, история, рабочая копия:

- **Хранилище (репозиторий)** — место где хранятся все версии файлов и история изменений
- **Commit** — зафиксированный снимок состояния файлов в определённый момент с описанием изменений
- **История** — последовательность всех коммитов с указанием автора, даты и описания
- **Рабочая копия** — текущее состояние файлов на локальном компьютере, с которым работает разработчик

3. Централизованные и децентрализованные VCS:

- **Централизованные** — единый сервер хранит все версии, разработчики получают рабочую копию с него. Примеры: CVS, Subversion (SVN)
- **Децентрализованные** — каждый разработчик имеет полную копию репозитория локально. Примеры: Git, Mercurial

4. Единоличная работа с VCS:

```
git init          # создать репозиторий
git add .         # добавить файлы
git commit -m "message" # зафиксировать изменения
git log          # посмотреть историю
```

5. Работа с общим хранилищем:

```
git pull          # получить изменения с сервера
# внести изменения в файлы
git add .
git commit -m "message"
git push          # отправить изменения на сервер
```

6. Основные задачи git:

Отслеживание изменений файлов, ведение истории версий, работа с ветками, слияние изменений от разных разработчиков, откат к предыдущим версиям.

7. Команды git:

См. таблицу ?? в разделе «Теоретическое введение».

8. Примеры работы с репозиторием:

Локальный:

```
git init
echo "hello" > file.txt
git add file.txt
git commit -m "first commit"
```

Удалённый:

```
git remote add origin git@github.com:user/repo.git
git push -u origin master
git pull
```

9. Зачем нужны ветки (branches):

Ветки позволяют разрабатывать новую функциональность или исправлять баги изолированно от основного кода. После завершения работы ветка сливается с основной. Это предотвращает попадание незаконченного кода в рабочую версию.

10. Игнорирование файлов при commit:

Создаётся файл `.gitignore` в корне репозитория, в котором перечисляются шаблоны файлов которые не нужно отслеживать:

```
*.log  
*.tmp  
node_modules/  
build/
```

Это нужно чтобы временные файлы, скомпилированные артефакты и конфиденциальные данные не попадали в репозиторий.

6 Выводы

В ходе выполнения лабораторной работы изучил идеологию и применение средств контроля версий. Освоил базовую настройку git, создание SSH и PGP ключей, настройку подписей коммитов. Настроил GitHub CLI (gh) для удобной работы с GitHub из командной строки. Создал репозиторий курса на основе шаблона и настроил его для дальнейшей работы.

Список литературы